

Pokemon Profiling: Insights from Data Exploration and Machine Learning

Jacob N

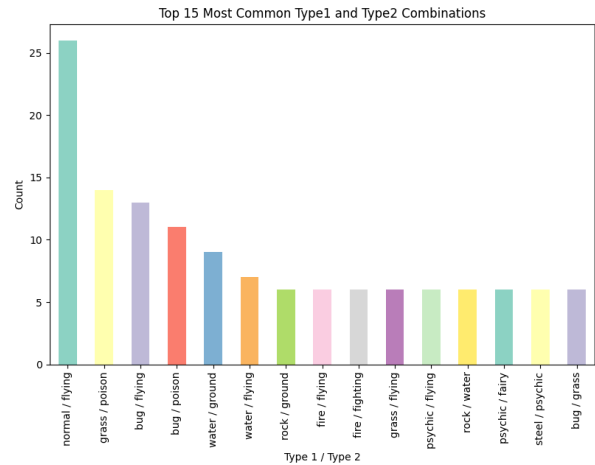
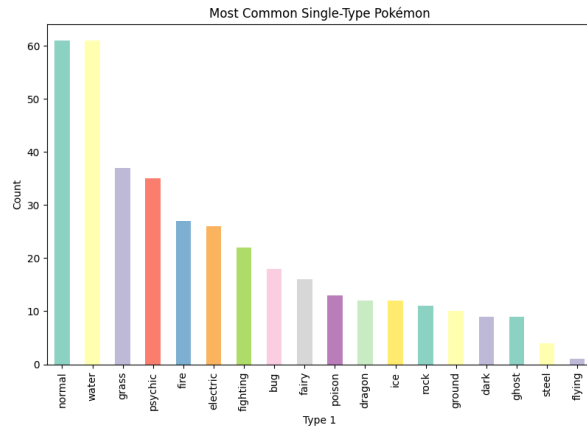
Introduction

The goal of this project was to explore the Pokemon dataset and see if we could predict how players classify pokemon in competitive and casual play. First, we explore the Pokemon data set and uncover trends on how pokemon has evolved generation to generation. After that, we do a full analysis on using K-means clustering and hierarchical data to try and classify the Pokemon into the prototypical RPG archetypes. We then test the model to see if it concurs with community opinion. Ultimately, we found that our algorithm was 88% accurate with this task. Finally, we created a logistic regression model to predict based on the stats whether or not a pokemon is a legendary. Here we say that the model is good at predicting legendary pokemon and when it does this prediction it is usually right. However, the model struggles at identifying legendaries, but when it does identify a legendary, it is usually correct.

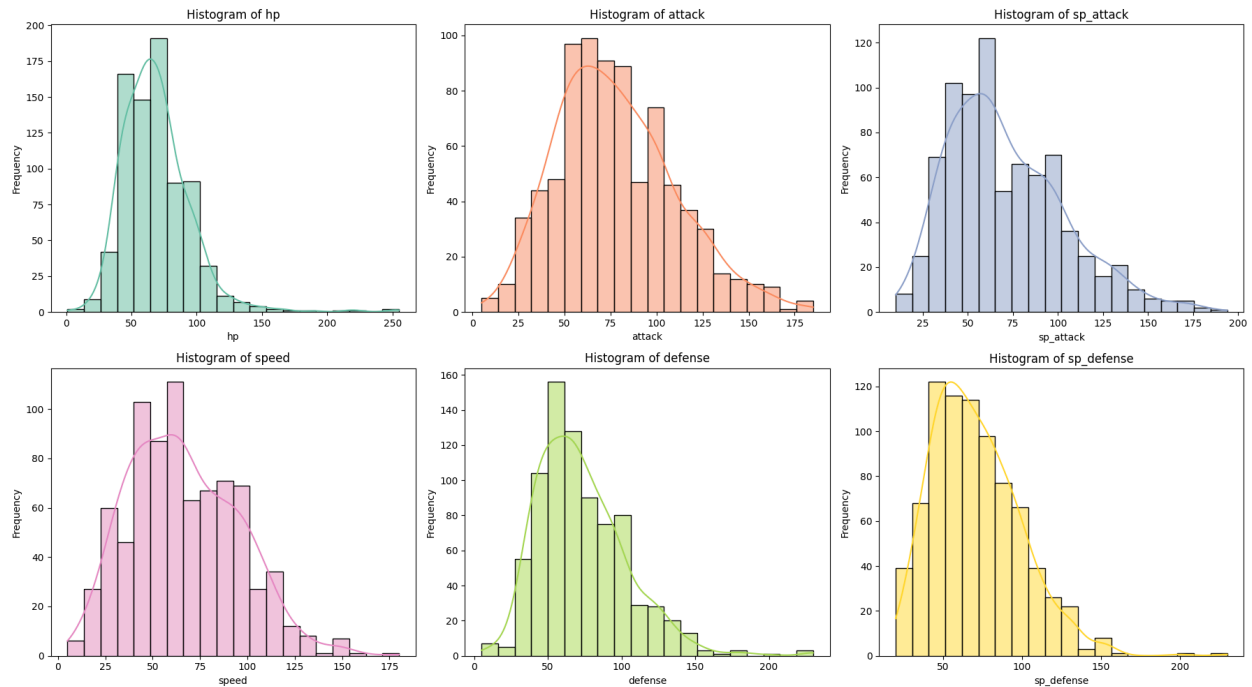
Data Exploration

We used the complete Pokemon Dataset, authored by Rounak Banik who compiled the information from the well known online Pokemon encyclopedia Serebii. The dataset contains all 802 pokemon from generation 1-7 (Kanto-Alola), and includes data on the Pokemon's name, base stats, types matchup, height, weight, shape classification, egg steps, experience points, abilities, stats, etc. This information itself comes from the national pokedex in the games themselves.

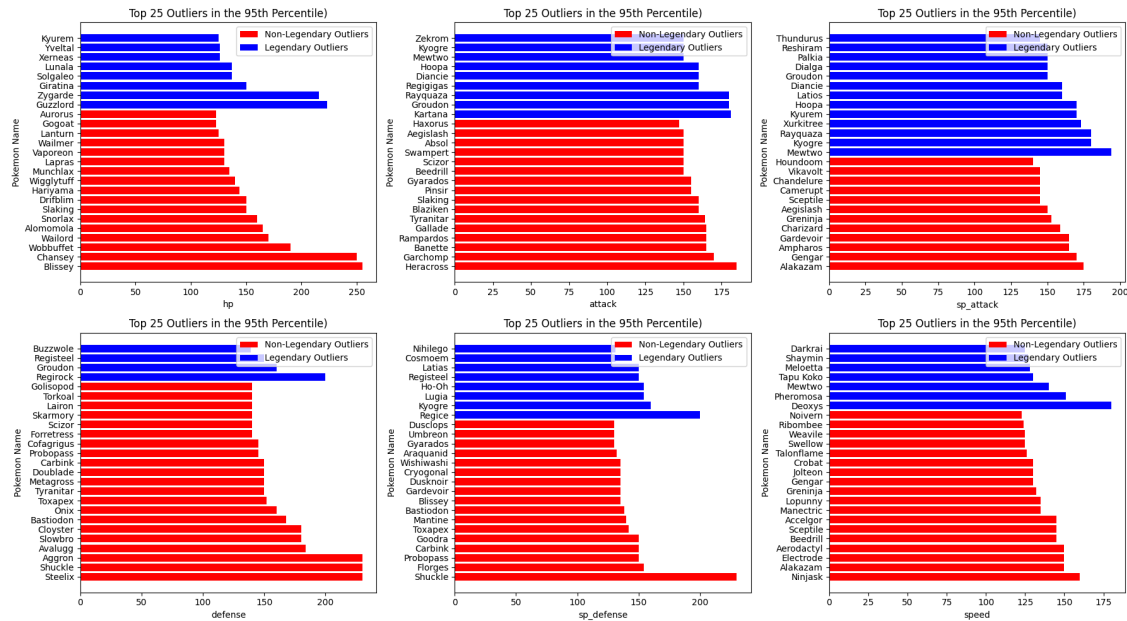
The first analysis we did was to check what were the most common Pokemon single and double typings. For single typings, we found that the top 5 were Normal, Water, Grass, and Psychic, while for dual typings they were Normal/Fly, Grass/Poison, Bug/Fly, Bug/Poison, and Water/Ground. Interesting to note is that while there is nearly no solo flying type pokemon, there is a massive amount of dual type pokemon.



Next, we wanted to begin to explore what the average pokemon's stats look like. For those that play Pokemon, note that we are only looking at base stats in this analysis and will be ignoring Evs and Ivs, which allow for some wild stat maximizing when utilized correctly. Here are the results we found examining for the averages:

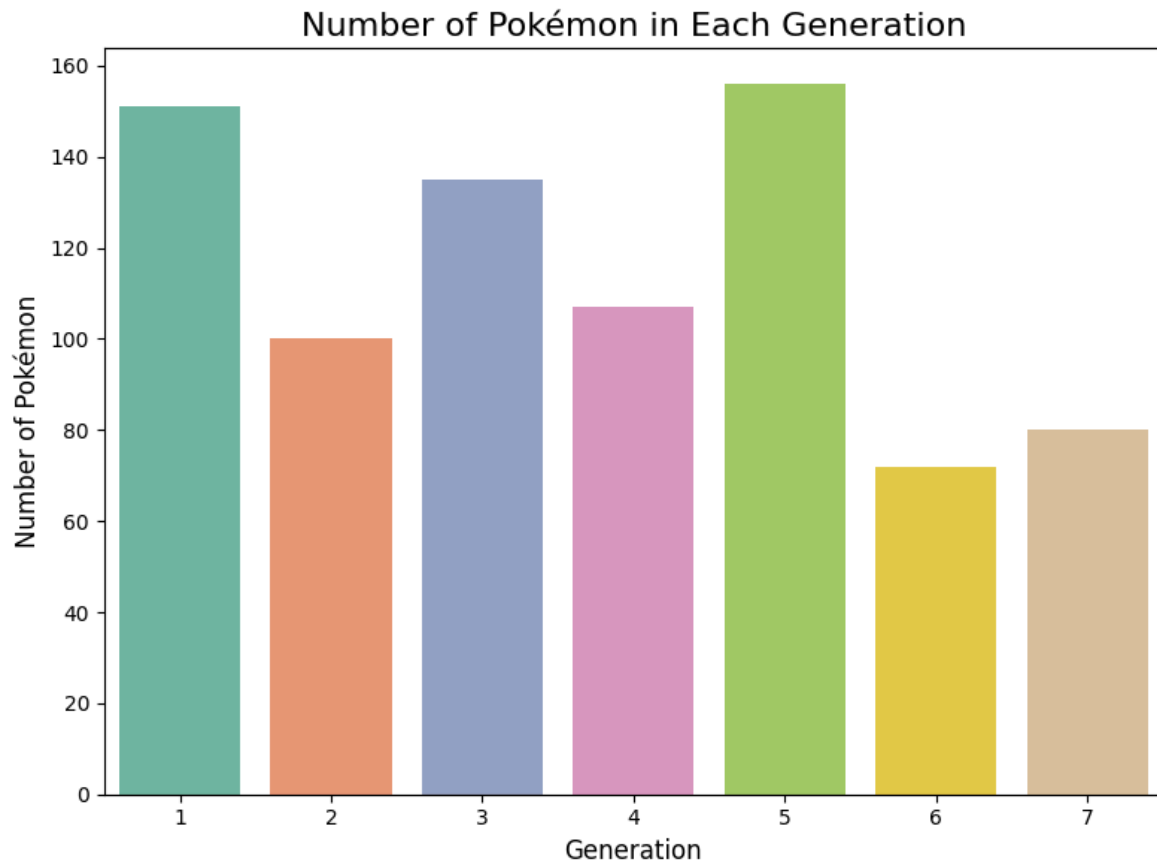


In general, the averages are all around 60. It is interesting to note however that both attack and special attack have more pokemon with high tier stats compared to the others. Zooming in on that section, we then examined what pokemon actually make up the upper 95th percent of each stat:



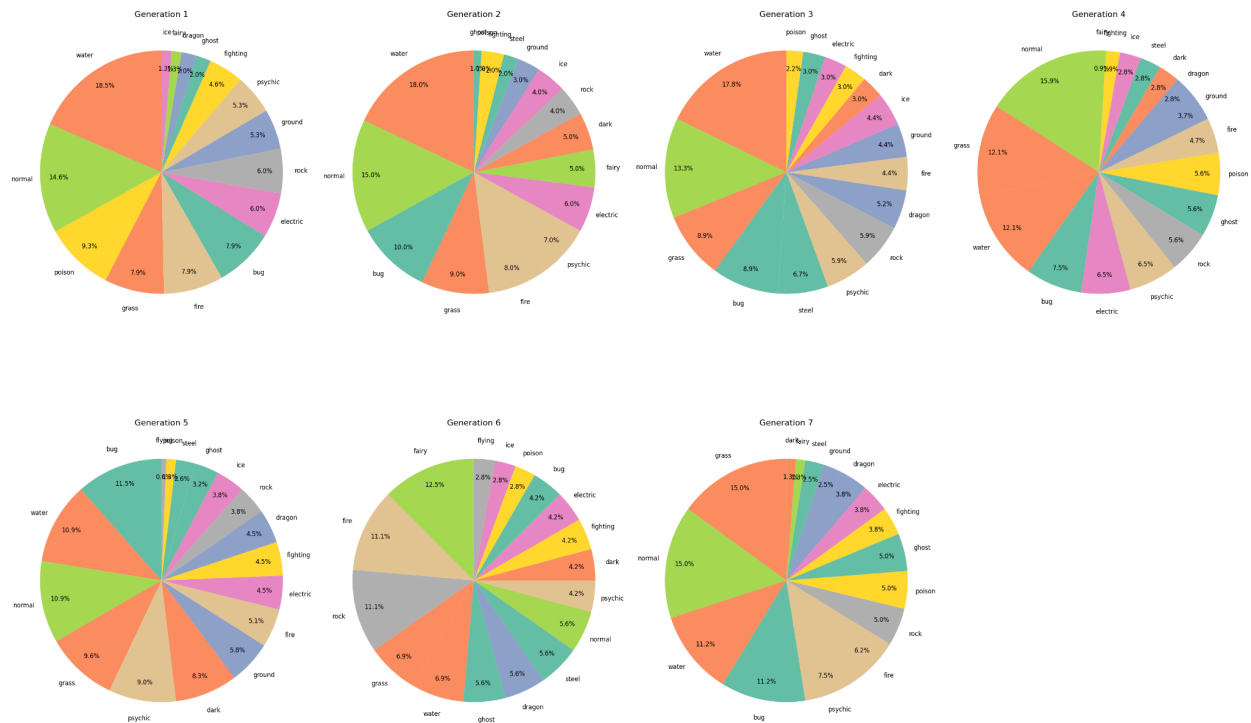
As we can see in blue, the median number of legendaries that show up in the top 25 pokemon of the 95th percentile for each stat category is 8, corresponding to nearly a third of the top 25 despite only making up 8.7% of all total population. The other interesting tidbit is for two thirds of the battle stats, non-legendaries take the top spot as the pokemon with the highest base stat, with Blissey, Heracross, Steelix, and Shuckle winning out for hp, attack, defense and special defence. In contrast, Mewtwo and Deoxys take the top spots for special attack and speed respectively.

Next, we wanted to explore how the Pokemon Co. populated each generation with Pokemon and what their distributions were. As we can see below, gen 1 and gen 5 both have the most pokemon added with 151 each. This makes sense as Unova was meant to pay homage to Kanto with its whole new slate of pokemon. Interestingly, within the first 5 generations, gen 3 has the third most newly added pokemon. Again, this makes Johto (gen 2) primarily made of left over designs from gen 1, and gen 3 marked a shift from the original gameboy to the gameboy advanced, freeing up more space and potential for new pokemon. Gen 6 has the least amount of new pokemon, but this also makes sense since it was the first game to be made in full 3d and they did not have any assets they could reuse.



We also wanted to see how the demographics of what types of pokemon were added changed over time. In general water, normal, grass, and bug almost always make up half of the pokemon types, as they are common and are prevalent in the early game. Specific shout outs go to generation 2 for adding steel and dark, and generation 6 for introducing fairy. In general ice, ghost, and steel types are the most rare generation to generation.

Percentage of New Pokémon of Type 1 Per Generation



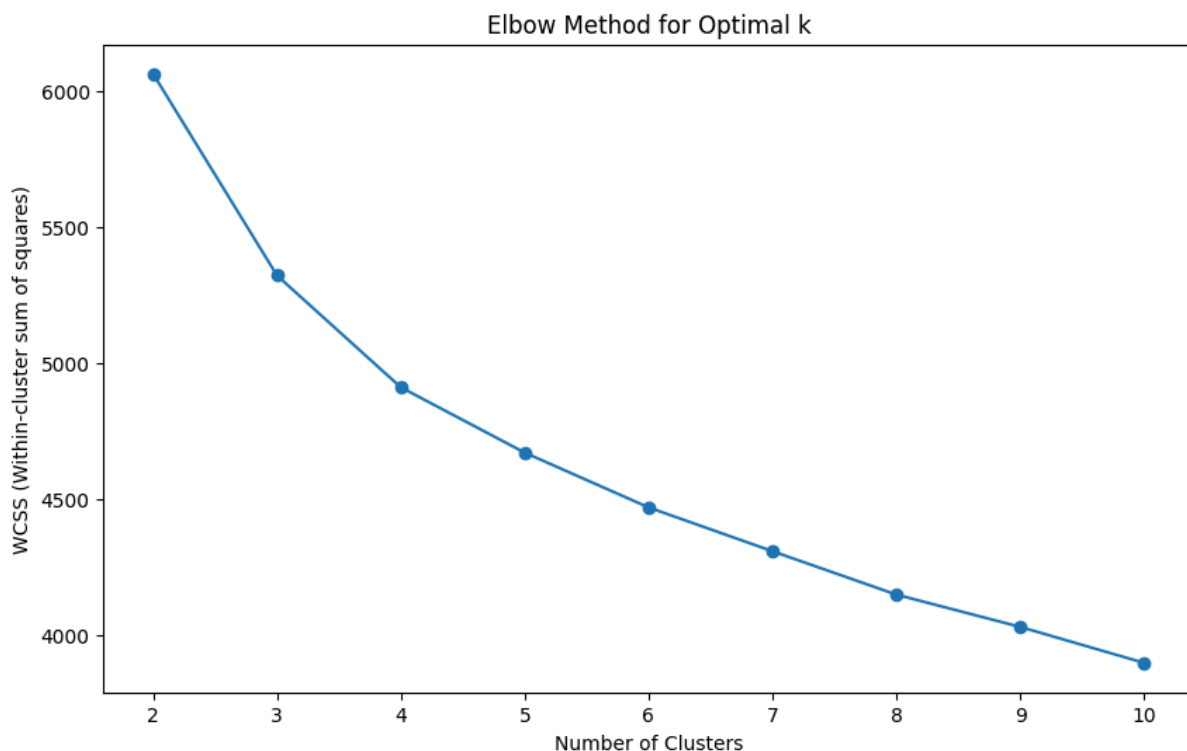
Clustering Pokemon Based on their Attributes

After exploring some of the basic properties of the dataset, we wanted to see if it were possible to extract some of the more complex categorical data about the pokemon. For example, typically in RPG gameplay there are general classes that characters fall into depending on their stats; for example, high defense and high hp usually lends itself towards being a “tank”, while having higher attack and high speed is more of a “glass cannon”. Our goal for this section is to see if we can use k-means clustering on the dataset to see how well we can categorize the Pokemon, and to see what clusters naturally appear in the data.

For the feature selection, we chose to use the standard stats—hp, attack, defense, etc—along with type and legendary status. Given the way the dataset was structured, we had to make sure that missing data entries were filled, and needed to convert the categorical types into numerical features using one-hot encoding. For simplicity, we also dropped the unnecessary columns from the working dataframe. Additionally, as we learned in the data exploration, although all stats technically have the same range, on average some base stats are less than others. As such, we scaled the stats so that all attributes would contribute equally when

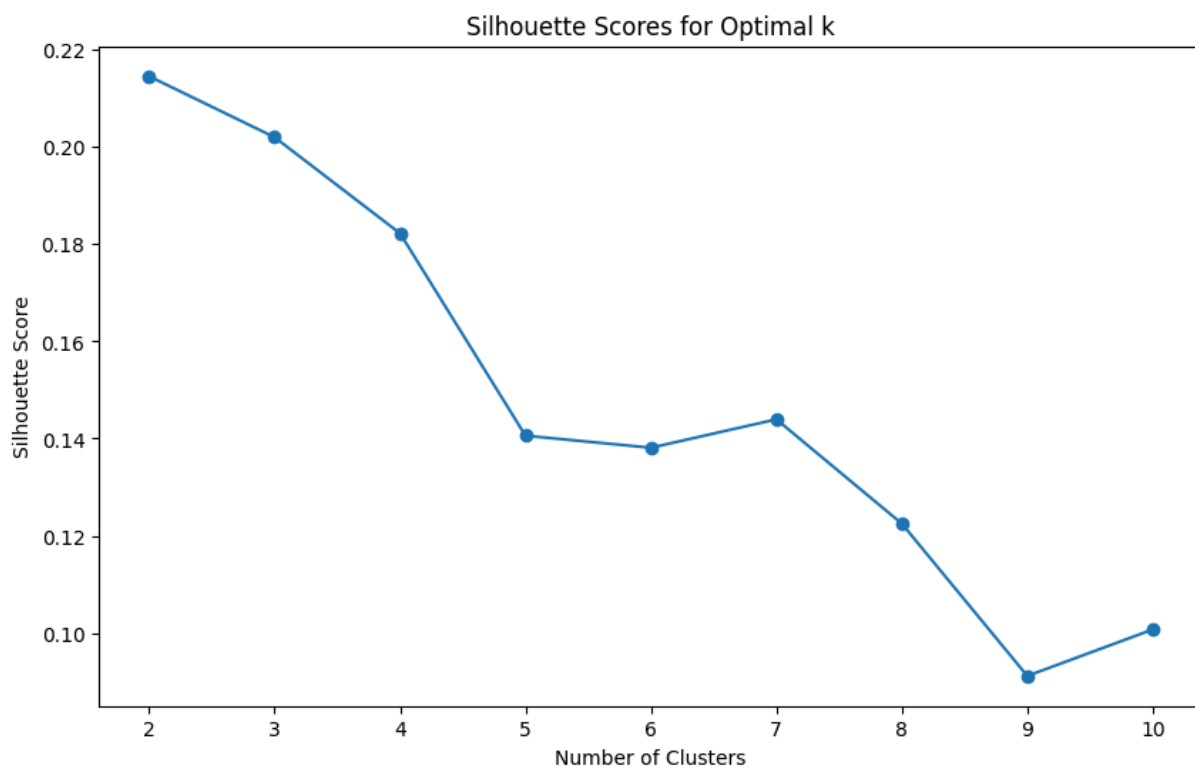
clustering. We then proceeded with k-means to cluster the Pokemon, where k was chosen based on Elbow and Silhouette score. After that we used PCA and the first 2 principal components for reducing the dimensionality of the data into 2d, allowing us to visualize it in a graph. Finally, we also did an alternative hierarchical classification using a Dendrogram.

First, as one of our metrics for determining the ideal number of clusters, we used the Elbow Method and made a WCSS (within-cluster sum of squares) plot. In general, the goal with Elbow is to pick a point that optimizes cluster size and WCSS. This usually occurs at the point WCSS begins to decrease at a slower rate.

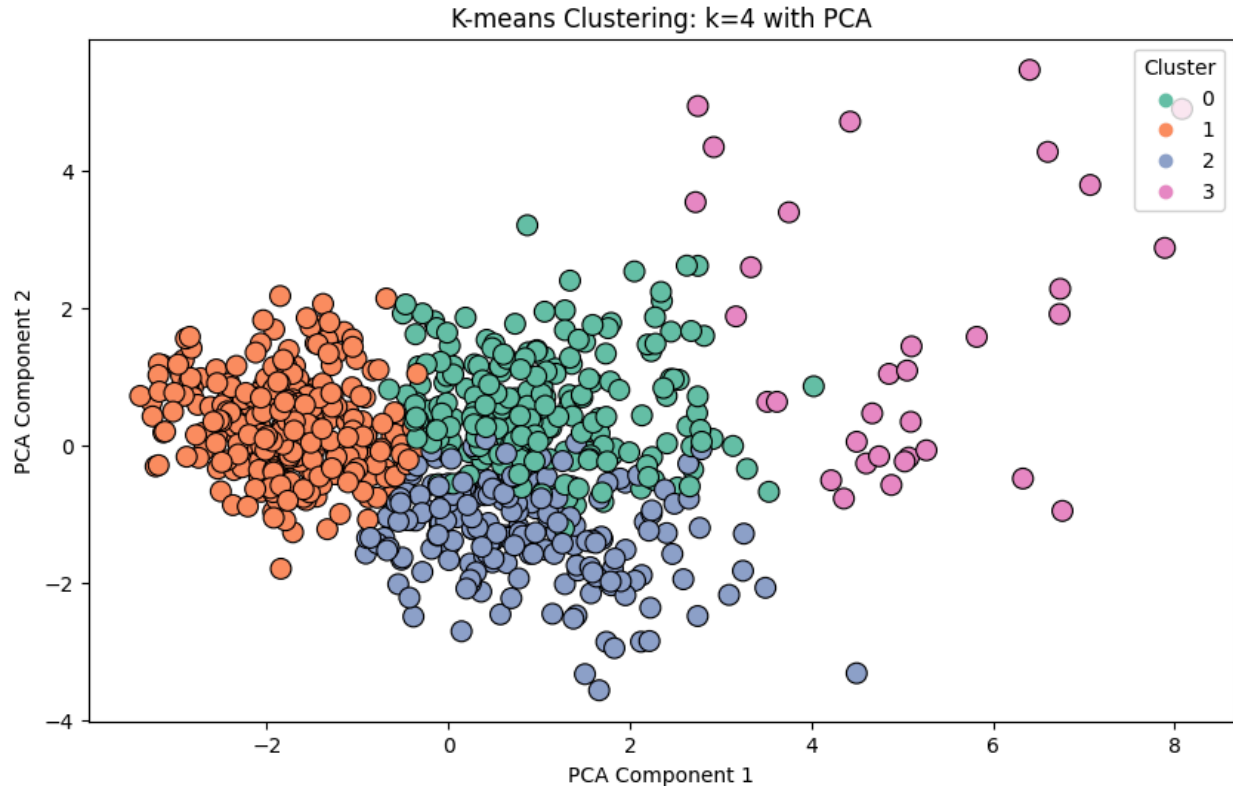


As we can see from the graph, we have a smooth concave up decrease in the curve from cluster sizes 2-10. The smoothness of the curve compared to other shapes would imply that while increasing cluster size does improve the fit of the data somewhat, it is not super drastic. Since WCSS measures the tightness of the clusters, a large drop would imply that the clusters are becoming more defined with increased K. Given our shape here there does not seem to be a clear winner.

In addition to Elbow, we also looked at the Silhouette Score which is used to measure the quality of the clustering and ranges from -1 to 1 where 1 implies well separated values, 0 implies overlapping and unclear clusters, and -1 implies points are assigned into the wrong cluster. As we can see from the plot, the score begins around 0.22 for $k=2$, then drops to 0.18 for $k=4$ and continues to decrease to 0.11 at $k=10$. A score of 0.11 for $k=2$ is valid, but leaves some room for hypothetical improvement. Additionally, the fact that the score drops after $k=2$ suggests that clusters beyond 2 are not that helpful and that choosing to go with a big k could be a bad choice since either the data has a lot of noise or the data points just do not naturally fit into the groupings. Ultimately, we decided on using a clustering of 4 as instinctually this felt like a realistic fit given domain knowledge of the dataset and generally the kinds of attribute patterns that are prevalent in Pokemon.



After deciding on the value of k , we then proceeded with k -means and used PCA as a dimensionality reduction technique to create our scatter plot across the first and second principal components. Principal components 1 and 2 are linear combinations of all the original features, meaning that they do not directly correspond to any one stat such as hp or attack; instead, they simply represent the largest variance in the data across all the features. Each point on the PCA scatter plot corresponds to a pokemon and the colors represent the cluster that was assigned to it according to k -means.



As we can see from the graph, we have 3 main clusters centralized tightly together around the origin (0,0). There is also a fourth, significantly more spread out, cluster in the positive part of the graph. For the most part, the boundaries for clusters 0,1, & 2 are well defined and have bounds, though there is some bleeding on the edge of cluster 0 and 2. Given that the clusters are well separated, this implies that k-means was able to find some distinct patterns in the features. From the generation, we found that the centroids of each cluster had the following stats:

	hp	attack	defense	sp_attack	sp_defense	speed
0	0.598194	0.383541	0.797036	0.198216	0.652783	-0.393859
1	-0.680808	-0.702219	-0.670004	-0.675782	-0.744652	-0.542515
2	0.147078	0.455814	-0.045964	0.651671	0.268502	1.168532
3	1.613717	1.296176	1.391194	1.040203	1.074223	0.450994

The centroids represent the mean values of the features for each cluster after k-means clustering. These values are in standardized form so they do not directly correspond to the raw value of the pokemon stats. This means that each score represents how much the mean value of that feature differs from the overall mean in terms of standard deviation; case in point, the value of 1.613717 for hp in cluster 3 means that the average HP in this cluster is 1.61 standard deviations above the mean for the entire dataset. A similar thing is true for negative values, for example the average attack in cluster 1 is 0.68 below the mean of the dataset. With this in mind, we can interpret cluster 0 as having high hp and defense with lower speed and attack.

Generally, this aligns with more tank-y pokemon. Next, cluster 1 pokemon on average have low everything. This could correspond to more “fragile” pokemon—it also makes sense that this is a large cluster since many Pokemon evolutionary lines begin as weaker and cutter creatures. Given how many evolutionary lines there are, it makes sense that these underdeveloped pokemon would take a cluster. In cluster 2, we see pokemon with high attack, special attack, and speed, but with low defense and special defense. This would correspond to “glass cannons” or “sweepers” which are able to do a lot of quick damage but cannot take a hit. Finally, we have cluster 3 which has above average stats for all categories. We can consider these as the most balanced or powerhouse pokemon. It also makes sense that this category is the smallest by far—on average only having a handful per generation—since they have the best overall stats. For a more concrete picture, we wrote a function that computes the top five Pokemon that are closest to the centroid of the cluster, which we can then use to examine their stats in the table below.

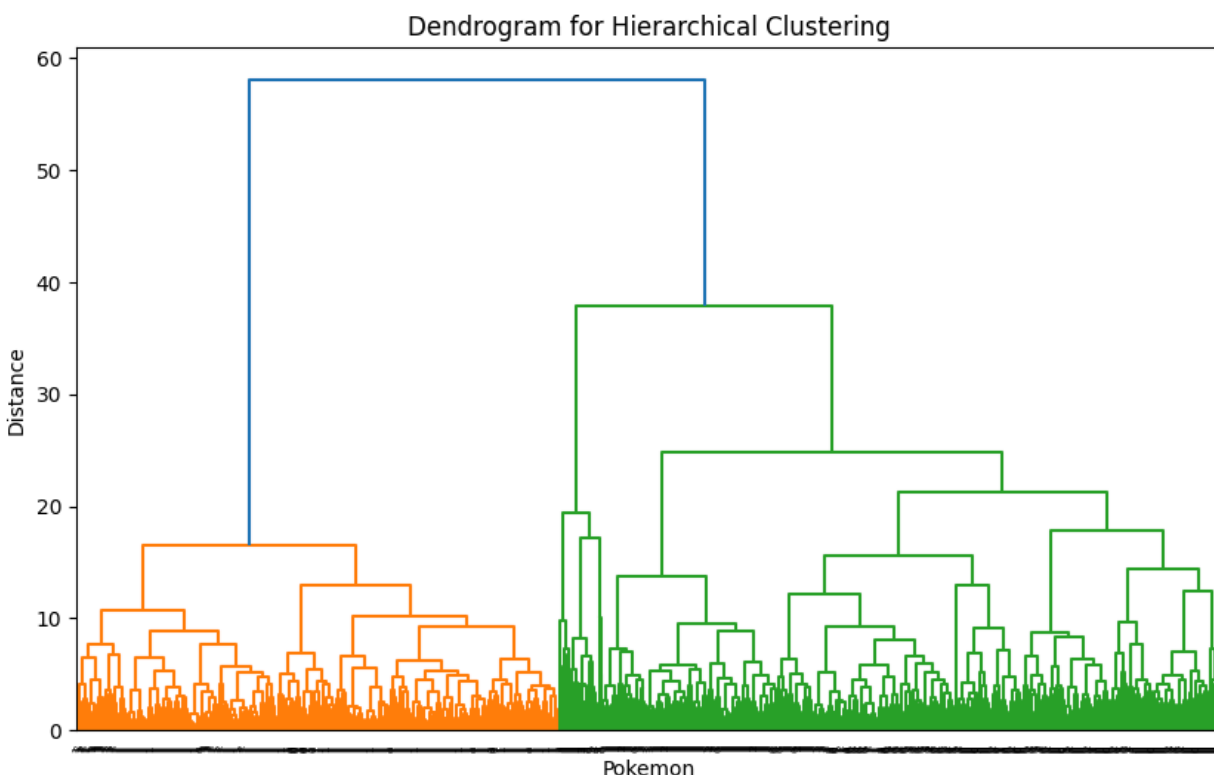
Top 5 central Pokemon for K-Means PCA Clusters

	Cluster	name	hp	attack	defense	sp_attack	sp_defense	speed	distance_to_centroid
0	0	Aromatisse	101	72	72	99	89	29	0.094759
1	0	Hitmonchan	50	105	79	35	110	76	0.134860
2	0	Carnivine	74	100	72	90	72	46	0.138740
3	0	Amoonguss	114	85	70	85	80	30	0.181876
4	0	Lurantis	70	105	90	80	90	45	0.194953
5	1	Servine	60	60	75	60	75	83	0.082425
6	1	Delcatty	70	65	65	55	55	90	0.102838
7	1	Ivysaur	60	62	63	80	80	60	0.104982
8	1	Combusken	60	85	60	85	60	55	0.133820
9	1	Dewott	75	75	60	83	60	60	0.145433
10	2	Vigoro	80	80	80	55	55	90	0.049367
11	2	Furret	85	76	64	45	55	90	0.192817
12	2	Gabite	68	90	65	50	55	82	0.250723
13	2	Breloom	60	130	80	60	60	70	0.257462
14	2	Mightyena	70	90	70	60	60	70	0.358180
15	3	Onix	35	45	160	30	45	70	0.449401
16	3	Aggron	70	140	230	60	80	50	0.590569
17	3	Snorlax	160	110	65	65	110	30	0.878990
18	3	Avalugg	95	117	184	44	46	28	0.920518
19	3	Cosmoem	43	29	131	29	131	37	1.543951

Given that the centroids present the average pokemon from each class, the results appear well founded from both a numerical perspective, and a subject domain perspective. Some notable

points: in cluster zero, Amoonguss is a popular gen 5 tank which fits in the category well. Hitmonchan is part of a split evolutionary line where it is intended to be the more offensive version—and it is compared to the split evolution—but as a whole it still falls under category 0 which is interesting. Shockingly, 4 of the 5 closest pokemon in the “weakling” category (Servine, Ivysaur, Combusken, and Dewott) are second second stage starter evolutions. This makes a lot of sense, as being starters their stats should be around average, which is what the centroid of Cluster 1 represents.

After calculating the PCA, we then also fed the info to create a linkage matrix and construct a dendrogram as seen below. Unfortunately, given the sheer size of the data the graph itself is not super useful unless you can actively zoom in and out of the small segments, so to help represent the data, just as before, we had a script pull out the center most parts of each cluster:



Top 5 central Pokemon for Dendrogram Clusters

	Cluster	name	hp	attack	defense	sp_attack	sp_defense	speed	distance_to_centroid
0	1	Snorunt	50	50	50	50	50	50	7.698172
1	1	Ditto	48	48	48	48	48	48	11.998336
2	1	Mareanie	50	53	62	43	52	45	13.847917
3	1	Pansage	50	53	48	53	48	64	14.035933
4	1	Pansear	50	53	48	53	48	64	14.035933
5	2	Necrozma	97	107	101	127	89	79	31.368615
6	2	Xerneas	126	131	95	131	98	99	32.689295
7	2	Yveltal	126	131	95	131	98	99	32.689295
8	2	Solgaleo	137	137	107	113	89	97	33.577622
9	2	Celesteela	97	101	103	107	101	61	35.164234
10	3	Typhlosion	78	84	78	109	85	100	12.621793
11	3	Infernape	76	104	71	104	71	108	15.871201
12	3	Simisage	75	98	63	98	63	101	21.866017
13	3	Simisear	75	98	63	98	63	101	21.866017
14	3	Simipour	75	98	63	98	63	101	21.866017
15	4	Poliwrath	90	95	95	70	90	70	12.770274
16	4	Nidoqueen	90	92	87	75	85	76	13.885333
17	4	Malamar	86	92	88	68	75	73	16.295138
18	4	Slurpuff	82	80	86	85	75	72	20.376359
19	4	Feraligatr	85	105	100	79	83	78	21.049003

What we found fascinating was that the dendrogram ended up converging on different categories for each cluster. For instance, Cluster 1 is now populated by the “weakling” pokemon. However, the pokemon found to represent the centroid are a third weaker than those found with k-means. Most notably here is Ditto whose entire gimmick is that it has awful stats until it transforms into the opposing pokemon and takes on theirs. Next, cluster 2 is composed entirely of legendary pokemon, specifically only from 2 regions: Kalos and Alola. Class 3 has two of the fully evolved fire type starters, and the fully evolved elemental monkey pokemon from generation 5. I would posit that this class is representing starter-like pokemon, that is those who are highest level and pure of type.

In a final test to see how accurate the classes were, I wanted to see if k-means clustering of the pokemon in rpg archetypes match how actual people who play pokemon categorize them. As such, I researched the most popular pokemon that people would consider to be in each class and checked if k-means agreed. The following table shows the results:

K-means Pokemon Cluster rating for Community defined Archetypes

	Pokemon	HP	Attack	Defense	Sp. Attack	Sp. Defense	Speed	Cluster
0	Skarmory	65	80	140	40	70	70	0
1	Steelix	75	125	230	55	95	30	3
2	Snorlax	160	110	65	65	110	30	3
3	Toxapex	50	63	152	53	142	35	0
4	Miltank	95	80	105	40	70	100	0
5	Geodude	40	80	100	30	30	20	1
6	Smeargle	55	20	35	20	45	75	1
7	Wimpod	25	35	40	20	30	80	1
8	Delibird	45	55	45	65	45	75	1
9	Magikarp	20	10	55	15	20	80	1
10	Deoxys	50	95	90	95	90	180	2
11	Aegislash	60	150	50	150	50	60	2
12	Salazzle	68	64	60	111	60	117	2
13	Archeops	75	140	65	112	65	110	2
14	Zoroark	60	105	60	120	60	105	2
15	Arceus	120	120	120	120	120	120	3
16	Palkia	90	120	100	150	120	100	3
17	Dialga	100	120	120	150	100	90	3
18	Giratina	150	120	100	120	100	90	3
19	Mewtwo	106	150	70	194	120	140	2

Looking at the table, the users and k-means agreed on the classification for 22/25 of the Pokemon, giving it an accuracy of 88%. Looking to where it failed, K-means claimed Steelix and Snorlax primarily as defensive tanks while the community voted on them as being the most balanced/powerhouse Pokemon. Referring back to the scatter plot, this would place Steelix and Snorelax along the green/pink divide where there is some mixing and ultimately some misclassification. However, both of these pokemon still score highly on community ranking for “tanks”, so it makes sense that this misclassification could happen. The final error was for the algorithm classifying Mewtwo as a sweeper while the community classified it as a balanced/powerhouse. Going back to the scatter plot, you can actually see the dot that represents Mewtwo as the one blue dot that is almost equally as close to the pink dot along the first principal component. Similarly to the other classification, many people in the community still play Mewtwo as a sweeper, it's just that the majority categorized him a step higher as an all around powerhouse as well.

What makes a Legendary, “Legendary” ?

Our last analysis is to see if legendary Pokemon have statistically significant differences in their stats compared to non-legendary Pokemon. To do this, we used a T-test to compare the mean stats between the two classes of Pokemon. We also built a logistic regression model to predict whether a Pokemon is legendary based on its stats. We first split our Pokemon data into training and test sets using the train test split function, with a test size of 0.2 and a random state of 42 for consistency. We then trained the Logistic regression model on the training set and evaluated the model using the accuracy score and classification report, along with creating a box plot to visualize the differences

T-test Report

	t-statistic	p-value
Stat		
hp	9.164280	4.127358e-19
attack	8.996885	1.663924e-18
defense	7.786876	2.129723e-14
sp_attack	12.568214	3.462358e-33
sp_defense	10.329799	1.452898e-23
speed	9.270647	1.684159e-19

The T-statistic is used to indicate how far apart the means of the legends and non-legends are using standard deviations. Larger values mean the differences between the two are more significant. As we can see, we have some large t-statistics, telling us that there are significant differences between the two means. We also have the p-values which help determine whether or not the means themselves are statistically significant, usually denoted as anything less than 0.05. For every stat, the p value is significant, so these tests hold.

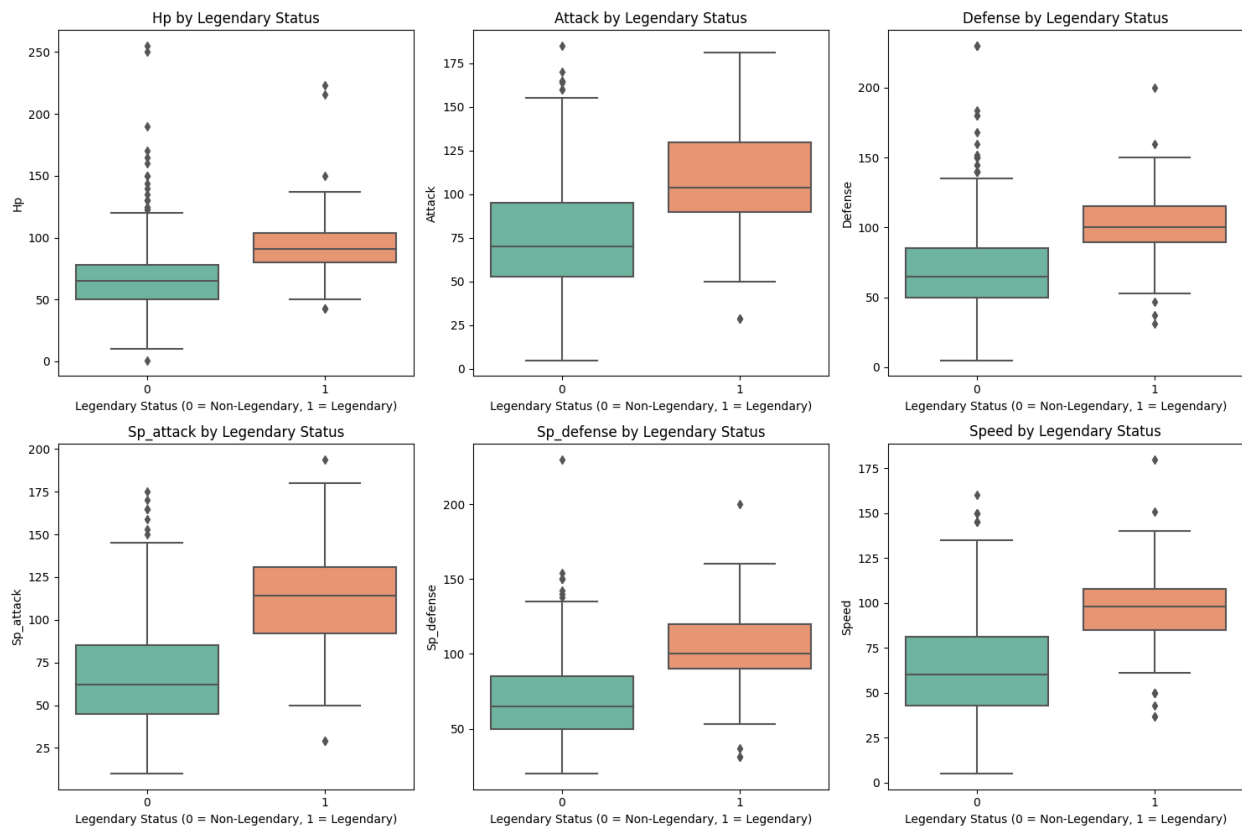
Logistic Regression Report

	precision	recall	f1-score	support
0	0.910256	0.993007	0.949833	143.000000
1	0.800000	0.222222	0.347826	18.000000
accuracy	0.906832	0.906832	0.906832	0.906832
macro avg	0.855128	0.607615	0.648829	161.000000
weighted avg	0.897930	0.906832	0.882528	161.000000

On one hand, the model does a good job at dealing with non-legendary pokemon and predicts them right often. Unfortunately, the model only properly identifies legends 22% of the time which is very low, but when it predicts that it is a legend it is correct 80% of the time. This is

most likely due to class imbalances; had I had more time to work on the project, I would have been able to implement some of the lessons we learned from the midterm, such as normalization, optimizing the features, using other models, etc. Still, a 22% guess rate is higher than guessing at proportional legendaries in the data set (8%), so you are more than twice as likely to get it right using the model rather than with random guessing.

Below, we also have our boxplots for the stats of the pokemon. The first thing to notice is that consistently, for all stats, the majority of the IQR for the legendaries is above the entire IQR for the regular pokemon.



We can also see that for the regular pokemon in all stats there are numerous outliers that lie in far above and in the range of the legendaries. The non-legendary pokemon that are high outliers are often called pseudo legendaries due to the stats being close to those of legendaries. Take for example Lucario and Metagross; though our model can correctly say that Metagross is not a legendary, it also claims that Lucario is one. This goes back to the plots at the beginning of the report that in the top 95th percentile of a stat, 66 percent is still just regular pokemon.

Conclusion

As we have seen, the Pokemon dataset has a lot of depth that can be used to uncover many interesting and unique trends both in how the franchise has tried to balance its gameplay

through the generations, and how players use pokemon in battle. Personally, we were appraised that the algorithms implemented were able to classify the pokemon fairly well both by their stats and by their higher order relatedness, such as starters and legendaries. I can also see many more extendable projects that can take place on the dataset. In particular, by adding in the moves that a pokemon could learn, there could be some interesting simulations using game theory and monte-carlo simulations to predict the results of pokemon battles. We could have also done some tests to try and predict the stats of pokemon using their type, region, and pokedex entry.